Irina H. Fylyppova

Senior Frontend Developer

📧 irina.h.fylyppova@gmail.com
🔗 GitHub | LinkedIn
🌍 Kharkiv, Ukraine | Remote only | UTC+2 (Kyiv), working across any timezone

Open to: Senior Frontend Engineer · Full-Stack Engineer (Vue / Node) · Frontend Architect · Developer Advocate · Technical Writer — contract or freelance

📁 **Project catalogue** — every project at full length

Professional Summary

Senior Frontend Developer with 8+ years building high-performance web applications — and the architecture underneath them. I take frontend estates end to end: at DGtek I was the sole frontend engineer across 14 repositories and ~72,000 lines of Vue, three applications over 100k+ customer and building records, with the entire data layer behind a Web Worker boundary and monthly revenue, churn and ARPU reporting moved off the server into IndexedDB cursors in the browser. 39 published npm packages, several of them shipped to third-party companies — one team's component became another company's website. My own products run to another 65,000 lines: Pagewright, a visual editor for multilingual sites on Nuxt 4, and a coordination platform on a Cloudflare Worker. I also write and teach — an open JavaScript course in three languages, on a rendering platform I built from scratch and release through a build gated by eight command-line checks.

Technical Stack

Frontend:
Vue 2/3 (Composition API, `<script setup>`) • Nuxt 2/4 • TypeScript • Vuetify 2/3 • Pinia • Vuex • Vue Router • Vue i18n • Vite • Vitest • Cypress

Performance & Data:
Web Workers • IndexedDB (with advanced indexing) • REST API • WebSockets • Real-time Synchronization • Geolocation APIs (Google Maps, MapLibre GL, Geoscape)

DevOps & Tools:
Git • GitHub Actions • GitLab CI • pnpm workspaces • Cloudflare Workers / D1 / KV • Cloudflare Pages • Firebase Auth • Firestore • Telegram Bot API • vite-ssg • Directus

Backend:
Node.js • Express • Cloudflare Workers (D1, KV, Cron Triggers) • SQL migrations • REST API design • Token verification and RBAC

Design & Testing:
Figma • SEO • Accessibility (a11y) • E2E Testing (Cypress, Playwright) • Unit Testing (Vitest) • Type checking (vue-tsc)

Professional Experience

Senior Frontend Developer — ITNET

Lviv, Ukraine | Feb 2025–Present · *Remote*

Eight projects on Vue 3 and TypeScript, from marketing sites to the platform below.

- **VD Express Platform** — a white-label booking core as a pnpm monorepo split by responsibility (`wizard-core`, `app-types`, `app-services`, `shared-ui`, `event-manager`). The core imports no host store: the application registers them through `registerStoresFor*()`, so one core serves several independent carriers. Step order is declarative too — `WizardStep` predicates rather than a hard-coded flow.
- **VD-Express & Site Constructor** — international coach booking: real-time seat availability over WebSocket with reconnection and ping monitoring, interactive cabin plans, door-to-door transfers on MapLibre, runtime-loaded themes for new brands without a rebuild, Firebase Auth, three locales.
- **BUSPORTAL** — a marketing site statically generated with `vite-ssg`: 20+ pre-rendered routes across four locales, Pinia state serialised into the markup, and the SEO layer extracted into a standalone composable.

Stack: Vue 3, TypeScript, Vuetify 3, Pinia, Vue Router, vue-i18n, Vite, Vitest, Cypress, MapLibre GL, Firebase Auth, WebSocket, pnpm workspaces

→ **All eight projects in the catalogue**

JavaScript & Vue.js Instructor — Ukrainian IT School

Kharkiv | Sep 2019–Sep 2025 · *Part-time, alongside client work*

Taught JavaScript and Vue.js to junior developers for six years, on a course I wrote and still maintain — and a shorter Vue course beside it.

- Fundamentals through to the awkward parts: functions as values, objects and constructors, references, prototypal inheritance, `this` and the ways it is lost.
- Vue 2 and Vue 3 with the Composition API, state management, REST APIs and project work.
- Students were onboarded onto commercial projects, which is the part that decides whether a course was any good.

The course outgrew the classroom and is now **Client-Side JS** — 164 lessons in three languages, described below.

→ garevna.github.io/js-lessons

Front-End Instructor — A-Level IT School

Kharkiv | 2017–2019 · *Part-time*

Where the course began. Two years of teaching front-end fundamentals, with the material written as it was taught — the first public release of it dates from 2017.

Senior Frontend Developer / Frontend Lead — DGtek Fibre Network

Australia | Sep 2019–Sep 2023 · *Remote*

Sole frontend engineer on a B2B provisioning platform for a fibre operator — 14 repositories, ~72,000 lines of Vue, three applications and 7 in-house npm packages, over 100k+ customer and building records. Four kinds of user: the operator's admins, the reseller ISPs selling on its network, the public checking an address, and the shareholders, who read the reports.

- **Architected the Web Worker data layer** shared by all three applications: a route/action message controller, credentials encrypted rather than left in localStorage, all API traffic, and a versioned IndexedDB cache of ten object stores with secondary indexes and migration-aware upgrades — which is what kept a heavy admin UI responsive and offline-tolerant.
- **Moved the reporting the shareholders read off the server**: monthly revenue, churn and ARPU are computed in a worker over IndexedDB cursors and drawn with Google Charts, removing a whole class of slow report endpoints.
- **Turned a component into a distribution channel**: coverage-check packages shipped to DGtek's reseller ISPs draw the operator's live polygons on sites DGtek does not control, resolve a visitor's address against them and quote a realistic connection time from that zone's settings. Coverage is edited in one place and every downstream site follows.

- **Packaged the address search**, which the worker boundary made possible: it geocodes against two sources — Geoscape for Australian addresses, Google Maps otherwise, with a fallback for addresses Google cannot resolve — then runs point-in-polygon on whatever coordinates come back to decide which coverage zone the building falls in. Self-contained behind a message port, so it shipped as a published package and ran on sites DGtek did not control.
- A fibre-haul cost calculator pricing a run to the nearest pit and exporting a client-ready PDF quote, and bulk import/export tooling with a validation pipeline.

Stack: Vue 2, Vuetify 2, Vuex, Web Workers, IndexedDB, Service Workers/PWA, Webpack, Google Maps API, Geoscape API, crypto-js, npm publishing

→ **Full case study** · **Catalogue**

Frontend Developer — Pineapple

ISP reselling DGtek's network | 2019–2022 · *Remote*

A Nuxt 2 public site, statically generated and composed from **10 npm packages I published**, with selective hydration through `vue-lazy-hydration` and navigation resolved from the CMS at runtime — plus the headless CMS itself (Vue frontend, Node/Express backend) storing landing pages as JSON with a publish/hide lifecycle.

Stack: Nuxt 2, Vue 2, Vuetify 2, Vuex, vue-lazy-hydration, PWA, Node.js, Express

---

Independent Products & Education

Full-Stack Developer — Pagewright

A visual editor for multilingual websites | 2026 · **live demo — the editor** · **the site it built** · source · *commercially licensed, not open source*

Pages are assembled from ready-made blocks, edited in place and published without touching code. My largest codebase: about 49,700 lines across 555 files — 143 components, a workspace package for the editor's own UI, a content API, and a second Nuxt application for the published site itself.

• **The published site is a separate application.** None of the editor's styles, plugins or settings can reach it, so a client's site can be deployed without the editor at all. One codebase renders it either way: on the server from the content API for a real site, and in the browser from the visitor's own IndexedDB for the demo — unsaved changes included, so a whole site can be built with nothing stored on a server.

• **Editing where the thing is.** Every block looks exactly as it does on the site and opens for editing on a click. New blocks are chosen from live previews rendered in the site's own colours and fonts, and land where the "+" was pressed. The whole page is also a list — drag to reorder, hide, jump to a block.

• **The site's look is data, not code.** Palette, UI kit, fonts, header and footer layouts and the site's own stylesheets are all editable, with CSS edited in the browser through CodeMirror. The editor has colour schemes of its own, so it never disappears into the site it is editing.

• **The editor opens in a role.** A content manager gets pages, menu, footer, banner and company data — and not the palette, the stylesheets or the technical settings. The person who writes the copy cannot accidentally restyle the site.

• **Multilingual throughout** — every text, page name and menu entry in each language, with the menu following a page when it is renamed. The demo ships in Ukrainian, English and German.

• **Built for a phone as well**: tools move into a bottom dock and panels slide up from the bottom rather than being a desktop layout squeezed narrow.

Stack: Nuxt 4, Vue 3, TypeScript, Vuetify 3, Pinia, vue-i18n, CodeMirror, Sortable.js, IndexedDB (idb-keyval), h3 and unstorage on the server, Service Worker, pnpm workspaces, Node 24

Full-Stack Developer — Bible

Reading app and ministry coordination platform | 2023–Present · kcc-bible.pp.ua · source

Started alone in 2023 as a Bible reader and grown into the tool a church runs its ministries on. Now developed with an AI coding agent — the practice above comes from here. About 25,000 lines; my only TypeScript-first codebase, and the one I would show to explain how I work today.

- **A serverless back end of its own**: a Cloudflare Worker verifies the caller's Firebase token and decides authorisation itself — roles and ministries in KV, messages, events, participants and materials in D1 behind five versioned SQL migrations, a nightly cron, a CORS allowlist, the bot token as a Worker secret. 3,600 lines of Worker TypeScript with 1,700 lines of tests beside it.
- **Permissions designed against lock-out**: reader, publisher, administrator, plus owners the admin page cannot modify, and no administrator can change their own role.
- **A rota with real operational logic**: recurring events with a different time per weekday and single-day overrides, two invitation modes, substitutes who agree in advance and step in automatically, escalation to the schedulers when both decline.
- **Two Telegram bots** — one publishing to the church channel, a service bot writing to ministers privately with inline buttons whose effect matches acting in the app.
- **A rewrite that kept its users**: a migration script mints a Firebase Auth account per legacy login, dry run by default, with `--check`, `--undo` and `--delete-legacy` as separate deliberate steps.

Stack: Vue 3, TypeScript, Pinia, Vuetify 3, Vite, PWA, Cloudflare Workers / D1 / KV / Cron, Firebase Auth, Firestore, Telegram Bot API, MapLibre GL, Playwright, Vitest, pnpm

---

Author & Platform Engineer — Client-Side JS

Independent | 2017–Present · garevna.github.io/js-lessons

The course from the two schools above, rebuilt as a product: 164 lessons in Ukrainian, Russian and English, on a rendering platform I wrote myself. Public and deployed since 2017 — garevna.github.io, vue-course (2018), js-simulator (2018), js-course (2019) and Client-Side JS today.

- **A rendering platform with no framework**: native custom elements with shadow DOM, a versioned service worker for offline reading, and web workers delivering content and icons.
- **An authoring language and its compiler**: lessons compile to components — replayable console and terminal sessions, quizzes, and code-repair exercises that run the reader's code in a sandboxed frame and score the repair.
- **Release gated by eight command-line checks**, including a byte-exact translation round-trip: every page is reassembled from its skeleton and message files and compared with the original before anything ships.
- **A serverless rebuild under a deadline**: when Glitch ended project hosting, the five practice back-ends twelve lessons depend on were rebuilt as one Cloudflare Worker, every route reconstructed from the request text printed in the lessons.

Stack: JavaScript (ES2022), Web Components, Shadow DOM, Service Workers, Web Workers, Webpack, Cloudflare Workers, Node.js tooling, Chrome DevTools Protocol

→ **Catalogue**

Core Competencies

Performance & data at scale — Web Workers and IndexedDB with secondary indexes and migration-aware upgrades; reporting computed over cursors in the browser instead of on the server; bundle splitting and lazy hydration.

Architecture — white-label and multi-brand systems configured at runtime; monorepos split by responsibility with dependency inversion, so one core serves several hosts; plugin-based reuse; state split into subject-specific stores.

Real-time — WebSocket synchronisation with automatic reconnection, exponential backoff and ping monitoring; multi-app data sync; offline-first fallbacks.

Back end — Cloudflare Workers with D1, KV and cron triggers; versioned SQL migrations; token verification and role-based access designed against privilege lock-out; REST APIs on Node and Express.

Accessibility & SEO — semantic HTML, ARIA, screen-reader testing; static pre-rendering with hydration; canonical, hreflang, Open Graph and generated sitemaps; dynamic meta tags for search engines and AI crawlers.

AI-Assisted Engineering

- **Agent-driven development with a verification loop:** a substantial share of my work is produced with an AI coding agent and none of it is trusted on sight — changes are exercised in a real headless browser over the Chrome DevTools Protocol, measured before and after, and reverted byte for byte when the measurement contradicts the claim
- **Harness before feature:** every new construct ships with a validator, so a malformed input is named on the command line instead of rendering as silent emptiness — the role an eval suite plays in an LLM product
- **Sandboxed execution of untrusted code:** reader-submitted code runs in an isolated frame and is graded by behavioural assertions rather than text comparison, because a real problem has several correct answers
- **Scoring instead of pass/fail:** an exercise can report that the code works and could have been written better, which is usually the truthful answer

Mentoring & Communication

- 10+ years mentoring junior developers, with the courseware public since 2017
- Onboarding students to commercial projects
- Direct stakeholder communication with C-level executives
- Technical documentation and code reviews — including documentation written for the non-technical people who use what I build

Education

Kharkiv National University named after V.N. Karazin
Higher Education (Mathematician)

Portfolio

39 published npm packages, and 87 public repositories of which 63 are deployed as live sites — Vue and JavaScript in roughly equal measure, with at least one release every year since 2017.

- 📦 **npm:** https://www.npmjs.com/~garevna
- 📚 **JS Lessons:** https://garevna.github.io/js-lessons/
- 🧠 **JS Quiz:** https://garevna.github.io/js-quiz/
- 🎨 **JS Philosophy Memes:** https://github.com/garevna/cv-resume/tree/main/media/js-memes
- 🔗 **GitHub:** https://github.com/garevna
- 💼 **LinkedIn:** https://www.linkedin.com/in/garevna/

Recommendations

Languages

- 🇺🇦 Ukrainian (Native)
- 🇷🇺 Russian (Fluent)
- 🇬🇧 English (C1 — Advanced)

Work Preferences

- ✅ Remote only
- 💰 Rate: Depends on scope and engagement — happy to discuss
- 📍 Timezone: UTC+2 (Kyiv)
- 🔄 Open to long-term engagements, complex projects, and architectural challenges